Beneficios de la IaC y la CaC

Estos son algunos de los beneficios que se obtienen al implementar filosofías como IaC y CaC:

- **Trazabilidad**: Esto se desprende del uso de sistemas de versionado para almacenar los ficheros de modelado y configuración, pudiendo, por ejemplo, volver a una versión anterior mientras se rastrea cual fue el cambio que introdujo un error determinado, reduciendo el downtime (tiempo sin servicio) y facilitando el proceso de detección y corrección del error.
- **Reutilización**: Al igual que en el desarrollo, una misma pieza de código genera siempre el mismo binario, un mismo script de IaC genera siempre una misma infraestructura. Esto permite desde instalaciones rápidas, migraciones por cambio de equipos o proveedor, hasta replicado de entornos enteros.
- **Agilidad y eficiencia**: La infraestructura como código automatiza la mayor parte de la administración de recursos, lo que ayuda a optimizar el ciclo de vida del desarrollo del software.
- **Alta escalabilidad**: Al utilizar IaC y CaC y trabajar con recursos de nube virtualizados, se pueden aprovisionar instancias de aplicaciones adicionales con facilidad y, literalmente, en poco tiempo. También se pueden liberar tan pronto como no se necesiten, lo que garantiza una rentabilidad incomparable.
- **Reducción de riesgos e infraestructura inmutable**: Como todos los entornos se aprovisionan de manera automatizada mediante mecanismos de CI / CD, no hay lugar para errores humanos. Esto garantiza la uniformidad del sistema en todo el proceso de entrega y elimina el riesgo de deriva de configuración, una situación en la que diferentes servidores contienen diferentes configuraciones y/o versiones de software debido a actualizaciones manuales realizadas en varias ocasiones a lo largo del tiempo.
- **Mejora de seguridad**: Si todos los servicios, ya sean de almacenamiento, redes, etc., son provisionados mediante scripts,



entonces se implementan de la misma manera cada vez que se ejecutan. Esto significa que los estándares de seguridad pueden implementarse de manera fácil sin tener que hacer que un rol de seguridad revise y apruebe cada cambio.

- **Mejor documentación**: Al igual que pasa con las aplicaciones, la verdad está en el código. En el caso de IaC y CaC solo tenemos que mirar el repositorio para saber el estado y la configuración de cualquier máquina en cualquier entorno. No hace falta meterse en un servidor a revisar y tener acceso a él. El repositorio nos dice exactamente lo que tiene una máquina.

Entornos de preproducción

Una vez hemos **completado el desarrollo y pruebas en local y en integración** de la nueva funcionalidad que se va a entregar en la siguiente versión, **necesitamos probar** también en algún **entorno lo más parecido al de producción**. Este entorno es el de **preproducción**. Normalmente solo se puede utilizar por los desarrolladores y/o equipos de QA y quizás por un pequeño grupo de usuarios para poder probar y asegurarnos de que esté todo correcto y que cuando se exponga en producción no haya ningún fallo.

Aunque este entorno de preproducción debería ser la réplica más exacta posible de producción, lo más común en la mayoría de casos es que los recursos destinados a este entorno sean inferiores por un simple tema de costes.

Normalmente se tiene un entorno de producción, pero se puede llegar a tener varios entornos de integración y algún entorno de preproducción (usualmente se tiene uno). Esta situación de tener tantos entornos, y que solo uno de ellos lo vayan a usar los usuarios finales significa que **las empresas tienen que invertir mucho dinero en entornos que no son productivos**. Las empresas tratan de reducir costes, porque no tiene mucho sentido tener 40 servidores productivos y 200 no productivos. Es por esto que **los entornos que no son de producción tienden a tener menos servidores, menos licencias de pago y, en general, menos recursos**.

Asimismo, cuando nuestra empresa tiene que integrarse con servicios de terceros, estos suelen darnos acceso a sus entornos de preproducción para poder probar las funcionalidades antes de pasar a producción. **Estos entornos de terceros suelen**, al igual que los nuestros, **estar dimensionados de forma distinta a producción**. Lo normal es que tengan un 50% de capacidad o menos que el entorno de producción. Esto tiene implicaciones a la hora de hacer pruebas de rendimiento, pues tendremos que ajustar la volumetría de dichas pruebas a la capacidad de nuestros entornos y de los terceros con los que nos integramos, extrapolando y prediciendo cómo será su comportamiento en producción.

En estos costes que tiene un entorno entran muchas variables y motivos por los que tendremos que buscar alternativas más económicas que simulen lo más fielmente lo que hay en producción para poder tener entornos preproductivos que sirvan para probar funcionalidades destinadas

a producción.

Como mínimo, hay que tener 2 entornos de preproducción siempre disponibles que cumplan las siguientes funciones:

- **Entorno de integración**: Se utiliza para la **integración del código desarrollado por todos los equipos** y la **realización de pruebas** de todo tipo sin que ningún cliente o persona ajena al desarrollo pueda entrar en este entorno. Este entorno es un **entorno seguro de desarrollo**, que actúa de primer filtro y red de seguridad de los programadores y que permite momentos de cierta inestabilidad. No es lo mismo que se caiga este entorno durante una hora y tengamos a 100 desarrolladores con un retraso para entregar lo que han hecho, que el entorno de producción caído durante 1 hora, y dejar de vender nuestro producto, por ejemplo. Esto no quiere decir que si rompemos este entorno nos podemos ir a casa y no pasa nada. **Lo ideal es que esté estable y levantado casi el 100% del tiempo**. Todas las pruebas se pueden hacer en este entorno sin miedo a las repercusiones. Por dar una cifra de ejemplo, este entorno, o entornos (porque puede haber varios) suelen estar dimensionados de forma que tienen el 50% de la capacidad de un entorno de preproducción o incluso menos.
    - *Como este entorno no es de producción, se trata de un entorno "preproductivo", término que engloba tanto al entorno de integración como al de preproducción, pero para distinguirlos, a este se le llama de integración. Su estabilidad, seguridad, propósito y recursos dedicados suelen ser diferentes.*
- **Entorno de preproducción**: Una vez superadas de manera exitosa las pruebas pertinentes, el código se desplegará en este entorno. Aquí podrían entrar equipos de QA (Quality Assurance) para **realizar tests extras**, gente de **negocio para poder comprobar la funcionalidad** de primera mano antes de que llegue a producción y **hasta un pequeño grupo de usuarios objetivo** para que den sus impresiones, ya que este entorno también puede servir de escaparate para captar potenciales nuevos usuarios. Por ello, **este entorno es el que va a tener más recursos destinados de entre todos los entornos "pre productivos" y en el que se va intentar tener todo los más parecido a producción,** pues después de pasar todos los filtros requeridos, el siguiente paso será el despliegue en producción. Dependiendo del proyecto, puede que sea importante que este entorno esté siempre disponible y no haya caídas, a diferencia del entorno anterior; por ello también es

posible que el cliente quiera que haya un equipo monitorizando el entorno 24/7 (siempre) y avisando si se detecta algún problema. Este entorno se suele llamar Staging, otros lo pueden llamar Sandbox y otros UAT (user acceptance testing). Este entorno suele tener un 50% de la capacidad del entorno de producción. Puede haber clientes en los que tenga un 100% de capacidad, porque los entornos sean prácticamente iguales, y otros casos en donde se tenga un 10% de capacidad.

El nombre de estos entornos no es lo más importante, pues la convención de nombres puede variar dependiendo del proyecto o del cliente. Lo realmente importante es tener entornos separados que cumplan estas funciones.

Además de estos dos entornos, se pueden tener más entornos si el cliente o el equipo de desarrollo lo cree necesario para poder realizar el trabajo; depende de las necesidades y presupuesto del proyecto.

La información almacenada en la base de datos de los distintos entornos de preproducción será distinta y estará separada por entornos. Esta separación se puede hacer teniendo instancias distintas por cada entorno o dentro de la misma instancia de base de datos, creando esquemas distintos o hasta incluso en el mismo esquema pero en tablas distintas por cada entorno. **La forma de separar los entornos en la base de datos dependerá del proyecto. Lo ideal y recomendable es que la base de datos de producción esté totalmente separada del resto de entornos, para que sean independientes a nivel de infraestructura**, y si tiramos una base de datos de pre, por pruebas de carga, que no afecte al entorno de producción. Es importante que el modelo de datos o estructura (DDL, Data Definition Language) sea igual en todos los entornos aunque los datos (DML, Data Manipulation Language) puedan ser distintos.

Con estos entornos hay que tener en cuenta que, al estar segregados y ser distintos, las personas que puedan usarlos también pueden ser distintas. No todo el equipo de desarrollo tiene por qué tener acceso a todos los entornos de preproducción; dependerá del proyecto. El único entorno preproductivo al que todos los desarrolladores siempre tendrán acceso será el primer entorno, que nosotros hemos dado a conocer como “entorno de integración”.



Entorno de producción

Haciendo un resumen de todo lo que hemos visto hasta ahora, hemos pasado de una idea de negocio a un desarrollo de un equipo, que se ha puesto en un entorno de integración junto con muchas otras ideas de negocio ya plasmadas en nuevas funcionalidades. Se han corregido ciertos comportamientos que no eran como se esperaban. Luego se ha promocionado la versión de la aplicación al entorno de preproducción, donde se han hecho más pruebas y un manager de negocio incluso ha probado como funciona, dando el visto bueno. Esto significa que en el siguiente “pase” a producción la versión llevará los cambios que hemos ido siguiendo desde el principio de este documento. Por fin hemos llegado al entorno de producción.

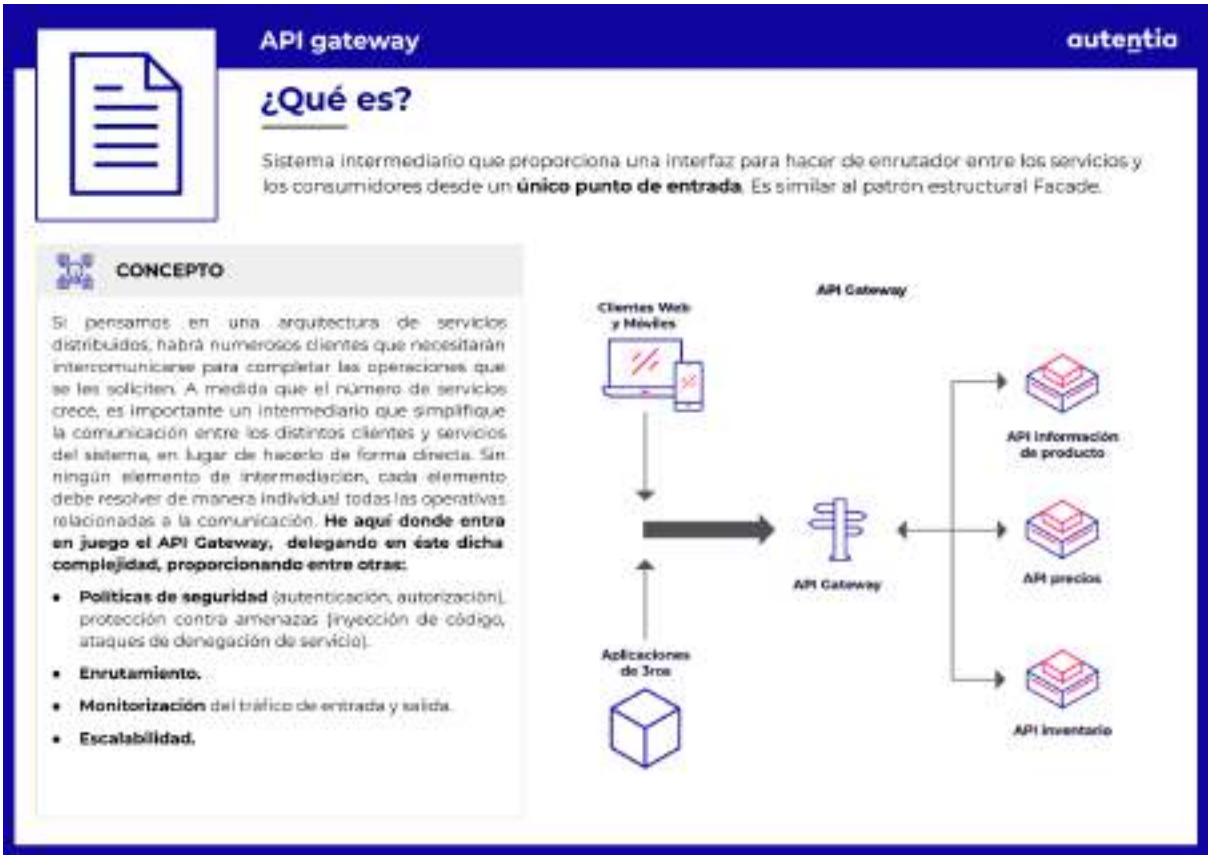
Infraestructura y servicios

El entorno de producción es toda la infraestructura y servicios relacionados con nuestras aplicaciones que usan los usuarios finales, y que nosotros controlamos de forma directa o indirecta. El entorno de producción no es el móvil del usuario donde se ejecuta nuestra aplicación, ni la página web que se le pinta al usuario final cuando accede a nuestra tienda. Para que se entienda mejor, tenemos los siguientes ejemplos de entornos de producción:

1. Los **servidores web** que sirven la página y exponen los servicios a los clientes.
2. La **CDN** externa que tenemos contratada, que cachea dicha página y elementos estáticos, como javascript, css e imágenes.
3. El **API gateway** que utilizamos para securizar nuestras APIs, monetizarlas y controlar su uso.
4. El sistema de **caché** distribuido que tenemos.
5. El servidor de **autenticación/autorización** que gestiona a los usuarios (login, registro, borrado de cuenta...)

6. La aplicación con la que **monitorizamos** el estado de la infraestructura y lanzamos alertas.
7. La aplicación con la que recogemos las **trazas** de producción y recopilamos información del uso de nuestros servicios.
8. La aplicación que utiliza negocio para ver si se están cumpliendo los **KPIs (Key Performance Indicator)**.
9. Todos los servidores que ejecutan de forma directa nuestras aplicaciones, con toda su **infraestructura, procesadores, memoria, red, discos duros**. Tanto si son **virtuales**, como si son **físicos**. Tanto si son **on premise**, como si son **cloud**.
10. La **base de datos** que contiene la información de producción, con los usuarios registrados, nuestros productos...
11. El **firewall** que da acceso a toda nuestra infraestructura.
12. Los **balanceadores** que redirigen la carga a las máquinas.

Si, para ofrecer algunos de nuestros servicios, necesitamos integrarnos con un tercero, estaremos usando SU entorno de producción, pero como no está dentro de nuestra organización y no tenemos control sobre el mismo, eso no se considera parte de nuestro entorno de producción. En nuestra mano está la correcta configuración de los mismos. Pero si esta configuración es correcta, y esos entornos fallan, habrá casos en los que nosotros fallaremos también sin posibilidad de poder hacer nada por nuestra parte salvo esperar a que se arregle.



Características de un entorno de producción

Los entornos de producción, por lo general:

- Cuentan con **más recursos hardware** que los entornos previos.
- La **seguridad es mucho mayor**. Pueden tener servicios de terceros para controlar el acceso, y filtrar el mismo.
    - Suelen estar **monitorizados** 24/7 con personas dedicadas a ello.
    - Suelen contar con un equipo de **intervención rápida**, con disponibilidad 24/7.
    - Suelen tener **alta disponibilidad**. Esto significa que su infraestructura está dividida en varias localizaciones físicas, separadas entre ellas varios kilómetros, y ante un desastre natural (como inundación, incendio o una fuerte tormenta) puede seguir dando servicio.

    - El **número de personas** que pueden acceder a ellos a distancia o de forma física y operar en los mismos es **muy reducido** y está muy controlado.
    - En el caso de exponer un API se utilizará **comunicación segura HTTPs** y se hará uso de un certificado digital.

- Es un **entorno muy delicado**, cualquier metedura de pata, dependiendo del negocio, puede tener costes muy altos. Todos los procesos deberían estar automatizados, por lo tanto.
- Es crítico **conocer** exactamente las **versiones desplegadas** de nuestras aplicaciones.
- La **forma de desplegar esas aplicaciones puede variar** frente a entornos previos para garantizar en todo momento la alta disponibilidad.
    - Sería ideal que la mayor parte del **proceso de despliegue** esté **totalmente automatizado**.
    - También sería ideal que la mayor parte del **proceso de rollback** a una versión estable anterior esté **totalmente automatizado**.



Alta disponibilidad

autentio

¿Qué es?

En inglés **High Availability (HA)**, se aplica cuando queremos tener un plan de contingencia sobre cualquier componente en caso de que se presente alguna situación irregular que impida la continuidad de los servicios.

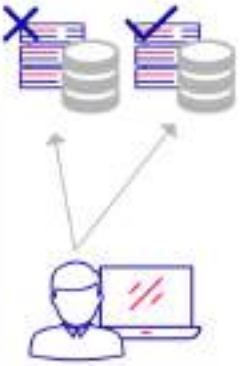
¿EN QUÉ CONSISTE?

Disponibilidad se refiere a la habilidad de los usuarios para acceder y usar el sistema. Se debe tener en cuenta que el tiempo de funcionamiento y disponibilidad no son sinónimos. Un sistema puede estar en funcionamiento y no disponible como en el caso de un fallo de red. **La criticidad del servicio es muy importante**, no podemos comparar la disponibilidad que debe tener un cajero automático (posiblemente 365 días 24x7) que otro servicio que solo esté disponible de Lunes a Viernes. Algunas **causas en la que podemos dejar de prestar servicio** ajenas a nuestra voluntad pueden ser:

- **Desastres naturales** (terremotos, inundaciones, incendios).
- **Cortes de suministro eléctrico.**
- **Errores operativos.**

Tener un sistema altamente disponible puede lograrse utilizando servicios cloud, los más conocidos pueden ser Amazon Web Services, Google Cloud o Azure. Necesitamos tener la capacidad de detectar un fallo en el servicio principal de una forma rápida y que a la vez, sea capaz de recuperarse del problema. La **monitorización** y **auditoría** de los componentes es fundamental para saber en tiempo real que está ocurriendo o como se comporta el sistema.

Si hablamos de un e-commerce se podría ofrecer el uso de **balanceadores de carga**, donde en vez de tener un único servidor, balanceamos la carga en varios servidores según la demanda que tengamos, de este modo, reduciremos las posibilidades de que el servidor principal colapse. Otras soluciones a tener en cuenta podría ser **dividir geográficamente la infraestructura en varias localizaciones físicas**, separadas entre ellas varios kilómetros, que ante cualquier desastre natural, pueda seguir prestando servicio.



Certificado

autentio

¿Qué es?

Documento virtual que contiene los datos identificativos de una persona física o de un sitio web y que está autenticado por un organismo oficial encargado de emitir y validar dicha información.

TIPOS

Cuando queremos identificar a una persona física, una entidad o un dominio de manera digital, hablamos de **certificados digitales**. Una **Autoridad Certificadora (AC)** será la encargada de firmar y emitir los certificados, verificando que quien lo solicite es quien dice ser.

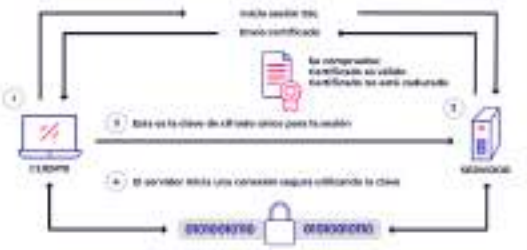
También podemos validar un sitio web a través de certificados. Se configura el servidor para que use dicho certificado y usando el protocolo SSL se cifra la información enviada al servidor. Esto es lo que comúnmente se conoce como **certificado SSL** debido a que este protocolo es el más extendido hoy en día. Los navegadores suelen incluir por defecto un repositorio de AC de confianza, pero si hay alguna AC en la que en principio no se confía (porque lleva poco tiempo en funcionamiento, por ejemplo), habrá que instalar manualmente el certificado en el repositorio del navegador. **Los certificados caducan** y esto se hace para asegurar que toda la información es precisa y demuestra una validez como propietario de confianza del dominio.

¿Qué ventajas nos ofrecen los certificados?

- Ahorro de espacio, tiempo y dinero.
- Confidencialidad y seguridad en las comunicaciones.

¿CÓMO FUNCIONA?

Se hace uso de claves públicas y claves privadas. En el caso de un sitio web, la clave privada permanece en el servidor donde se ha configurado el certificado. Cuando un navegador web desea establecer una nueva conexión, el servidor comparte la clave pública con el cliente para establecer un método de cifrado y el cliente confirma que reconoce y confía en la entidad emisora del certificado. Este proceso inicia una sesión segura que protege la privacidad y la integridad del mensaje.



DevOps, DevOpsSec y demás siglas

DevOps es una combinación de las abreviaturas "**Dev**" (desarrollo) y "**Ops**" (operaciones). Podríamos definirlo como un marco de trabajo en el que tanto el departamento de desarrollo como el de operaciones colaboran aportando ideas, pruebas prácticas y el uso de nuevas tecnologías para conseguir mejorar los procesos de desarrollo del software. En ningún caso, se trata de aumentar la responsabilidad de los desarrolladores, haciéndoles responsables de funciones del departamento de operaciones.

- Últimamente también se están oyendo las siglas **DevOpsSec**, que implica la participación de un nuevo actor, el departamento de seguridad para forzar a que se piense en la seguridad desde el principio, evitando el enfoque tradicional de auditar la seguridad del software o la infraestructura únicamente cuando se sale a producción. Algunas de las formas que tiene el departamento de seguridad de incorporar la seguridad durante todo el ciclo de desarrollo son:

- Incorporar el escaneo de vulnerabilidades en el ciclo de desarrollo con herramientas como Anchore o SInk.

- Estandarizar y automatizar las políticas de seguridad concediendo a cada servicio el privilegios mínimos necesarios para poder ejecutarse.

- Centralizar la identificación de usuarios y control de accesos con servicios como IAM (Identity and Access Management) de AWS.

- Incorporar dentro del pipeline de CI, herramientas que realicen análisis estático de código centrado en la seguridad con herramientas como Kiuwan Code Security o el propio Gitlab.

- Utilizar entornos aislados y que usen lo imprescindible para realizar su función como los contenedores de docker, nos permite reducir la superficie de ataque.

- Automatizar las actualizaciones de seguridad y parches de vulnerabilidades conocidas.

- Hacer cumplir las políticas de seguridad, modificando los ficheros de

configuración de los entornos que se despliegan automáticamente, evitando errores manuales.

- Uso de cifrado punto a punto y APIs seguras.

Todas las técnicas o medidas expuestas son compatibles con la realización de Pentesting o pruebas de intrusión tanto a la aplicación como a la infraestructura.



Parte 2

Piezas básicas de la infraestructura

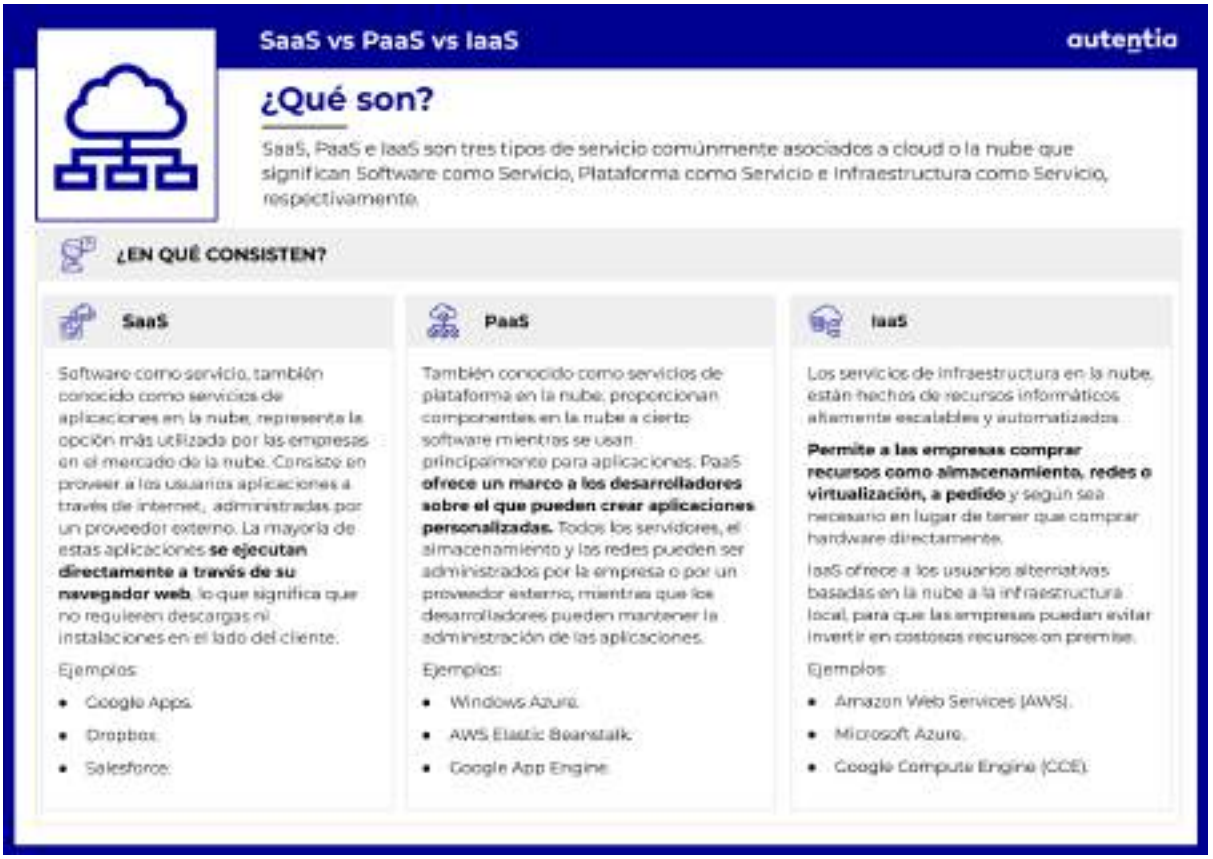
¿En la nube o no? Ese es el dilema...

En el capítulo anterior hemos visto todas las piezas que se utilizan a la hora de desarrollar software, así como el camino que éste sigue hasta alcanzar el entorno productivo, donde los usuarios finales pueden hacer uso de nuestra aplicación. Esto es así cuando las aplicaciones se llevan al entorno de producción. A esta acción de mover una aplicación de un entorno a otro (normalmente más cercano al usuario final y más lejano al desarrollador software) se le conoce también como **promocionar**. Cuando una aplicación es promocionada al entorno de producción, la versión anterior de dicha aplicación es sustituida por la nueva versión.

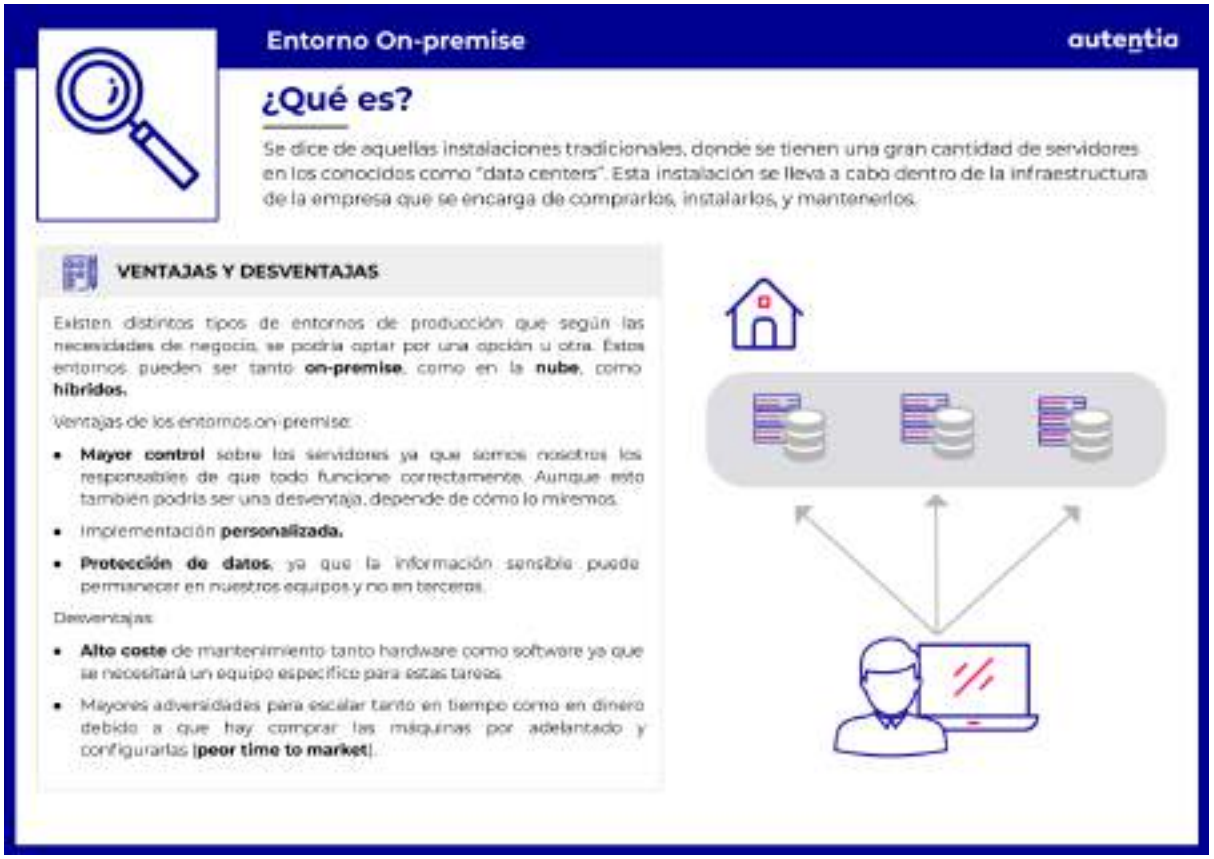
"Antes del advenimiento de la nube, los entornos eran todos on-premise con resultados en la gestión de grandes éxitos y grandes fracasos."



Estos entornos de producción pueden ser tanto on-premise, como en la nube. También existen entornos híbridos.



- Entornos **on-premise**: se dice de aquellas instalaciones tradicionales donde se dispone de un montón de servidores en los conocidos como “data centers”, que son salas llenas de servidores, con acceso restringido. Estos servidores son propiedad de la compañía que se encarga de comprarlos, instalarlos y mantenerlos. Es una opción que requiere un desembolso inicial grande.



- Entornos **cloud**: en este caso nuestras aplicaciones se ejecutan en ordenadores que no son de la propiedad de nuestra compañía sino de un proveedor de nube, que es quién compra esos ordenadores, los mantiene y se encarga de la distribución física de los mismos, de la conectividad de redes, del control de accesos, etc.



- Entornos **híbridos**: se trata de aquellos que **son una mezcla**. Parte de la infraestructura es de la empresa y parte está en la nube.



Ventajas e inconvenientes de los entornos on premise

"On Premise ofrece mayor control. Nosotros somos responsables directos de que todo funcione... A un mayor coste y peor time-to-market"

Ventajas:

- El **mayor control** que tenemos sobre los servidores, su conectividad, su seguridad, su mantenimiento. Somos nosotros los responsables de que todo funcione como debe.

Inconvenientes:

- Tener más control, que en principio es una ventaja, es muy costoso, ya que tenemos que disponer de departamentos dedicados a ello: mantenimiento y actualización de los sistemas, sustitución de piezas que se rompen, solucionar problemas de conectividad, etc. **La principal desventaja de esto es el gran coste que conlleva**.
- Estos sistemas no escalan de forma automática. Para escalar hay que comprar las máquinas por adelantado y configurarlas, lo que acarrea un **mayor coste y mucho más tiempo**, o lo que es lo mismo, **peor time to market**.

Ventajas e inconvenientes de los entornos cloud e híbridos

Ventajas:

- No tenemos que comprar máquinas, solo configurarlas y usarlas, lo que en principio supone un **menor coste**.
- Se paga por lo que se usa, no tenemos que ser previsores y comprar de más, volvemos a tener **menor coste** por lo tanto.
- Estas infraestructuras **escalan de forma automática**, tenemos mejor time to market, y mejor respuesta ante subidas de tráfico, campañas publicitarias etc.
- Los **servidores se actualizan solos** aplicando parches con actualizaciones y mantienen conectividad sin que tengamos que hacer nada.
- La configuración de acceso a los servidores se simplifica gracias a las facilidades que ofrece el proveedor de cloud. Eso implica lógicamente **una reducción de costes** en este sentido.

Inconvenientes:

- Si no se configuran bien y se ponen límites **pueden disparar el gasto de forma descontrolada**.
- En la mayoría de los casos tenemos **menor control sobre los servidores**.
- Si ocurre un fallo, en muchas ocasiones no podremos hacer nada salvo esperar a que el proveedor de nube lo arregle.

“Cloud representa menor coste, pago por uso y escalado automático. Si no se controla, se dispara el uso de recursos y aumenta el gasto.”

Los sistemas híbridos pueden tener todas las ventajas e inconvenientes de las otras dos opciones. Tendremos que optimizar los servicios que irán en la nube para maximizar las ventajas, lo mismo que para las instalaciones on premise deberemos optimizar su uso para no incurrir en altos costes o no poder escalar de forma rápida.



Servidor Web

Un Servidor Web es un software que se utiliza para **procesar peticiones HTTP o HTTPS** y servir las páginas web o el contenido estático (ficheros Javascript, hojas de estilo, imágenes, fuentes, etc.) al dispositivo que le esté realizando esas requests (cliente). Los más utilizados actualmente son Apache, Nginx y Microsoft IIS. Es frecuente que a la máquina que tiene instalado este software, junto con los ficheros de la página web (documentos HTML, imágenes, CSS y ficheros JavaScript) se le llame también “Servidor web”.



HTTP

Ahora bien, ¿cómo funcionan las peticiones a estos servidores?, ¿qué es HTTP? HTTP o Hypertext Transfer Protocol es la base de la comunicación de datos para la World Wide Web. Es un protocolo de aplicación para sistemas de información hypermedia, lo que significa que es un medio de

transmisión de información que puede incluir gráficas, audio, vídeo, texto puro e hyperlinks (comúnmente llamados links).



HTTP es un protocolo petición-respuesta (request-response), lo que quiere decir que será el cliente (dispositivo navegando por internet) el que realizará peticiones al servidor y este simplemente, responderá a estas peticiones. Este protocolo se basa en varios tipos de operaciones o **verbos HTTP**. Básicamente, cuando se hace una petición hay que especificar al servidor qué tipo de operación queremos realizar: ¿necesitamos visualizar una página o recoger información? Lo indicaremos usando el verbo **GET**.

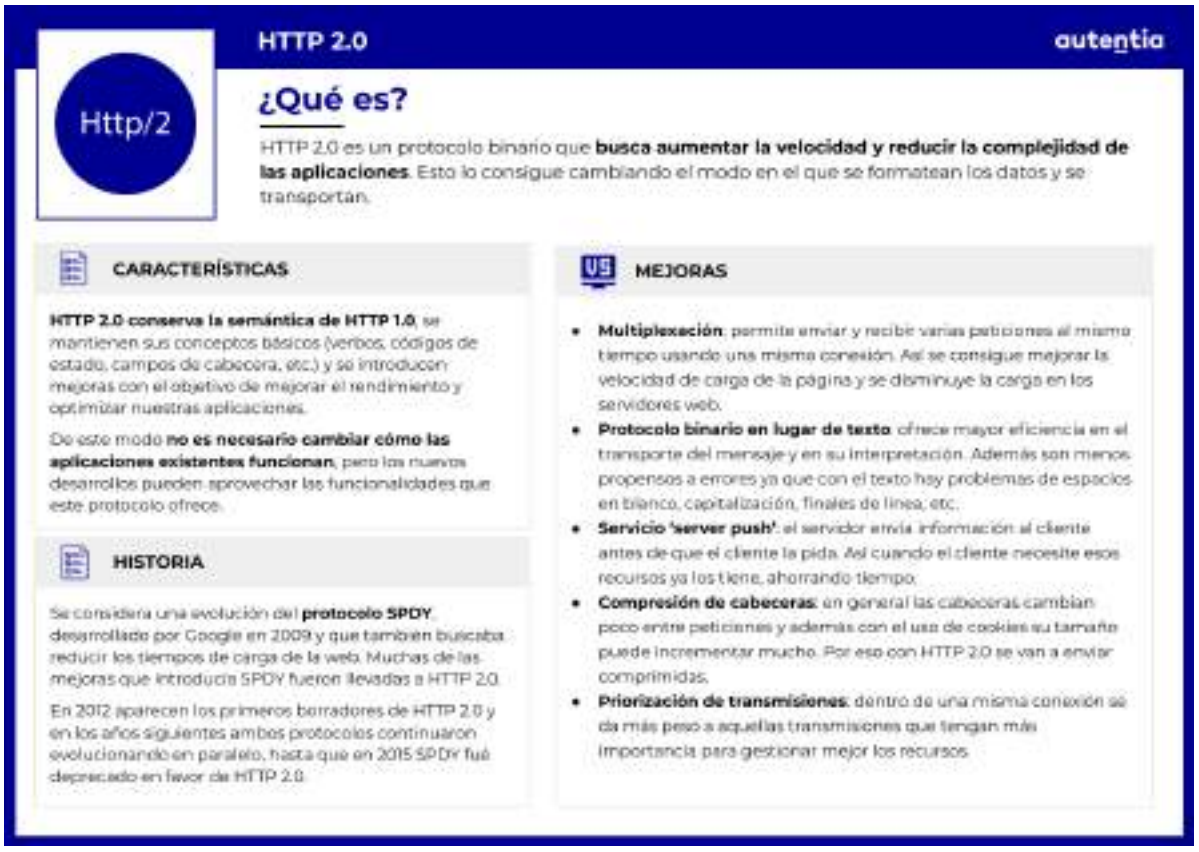
¿Queremos mandar nueva información al servidor, como cuando se da de alta un usuario nuevo o un post en una red social? Usaremos el verbo **POST**. ¿Queremos modificar alguno de nuestros datos de usuario en alguna página o editar un comentario en una foto? Realizaremos la petición con el verbo **PATCH** o **PUT**. ¿Queremos borrar un comentario o una foto? Para ello utilizaremos **DELETE**. Hay más verbos además de los ya mencionados pero estos son los más utilizados.

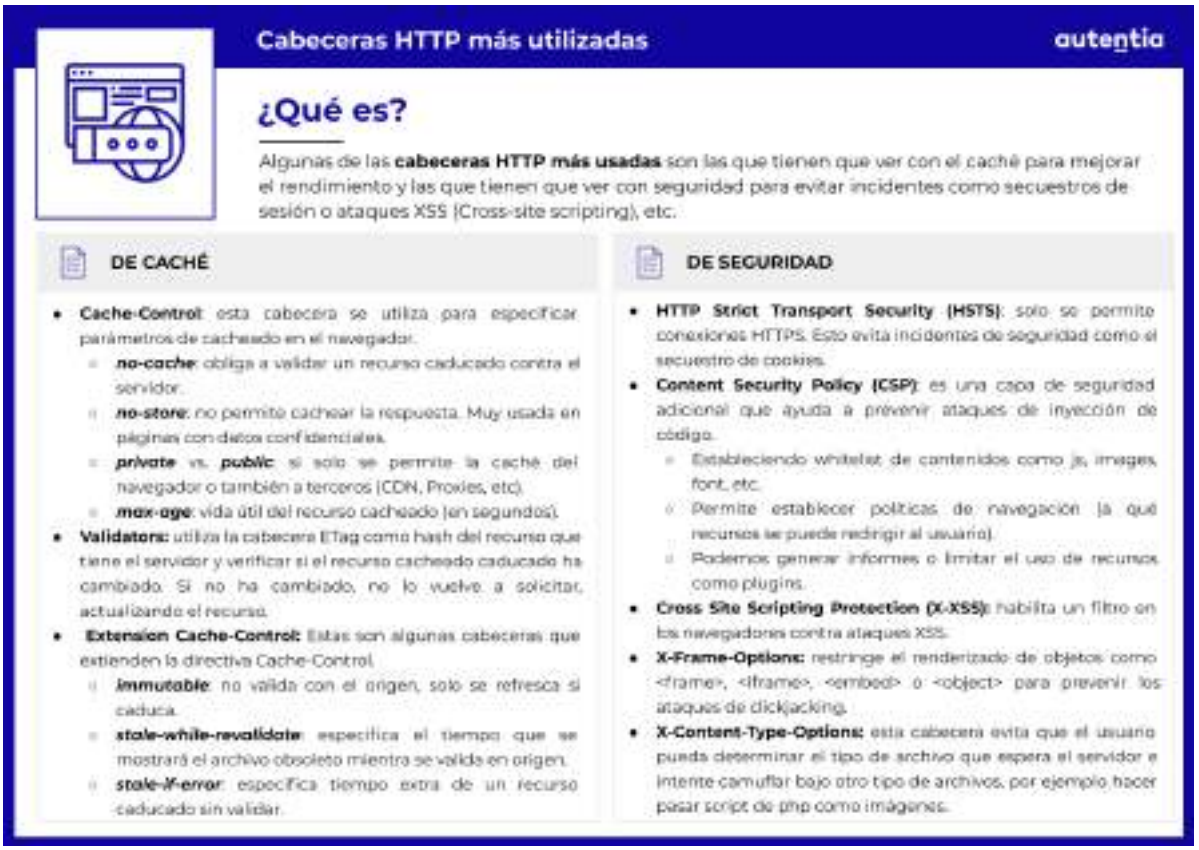
Aparte de indicar el verbo tenemos que indicar cuál es el recurso al que queremos acceder. **HTTP expone recursos** que son localizados a través de

lo que se denomina URL (Uniform Resource Locator), por ejemplo, si hacemos un GET de *https://protectoranimales.com/pets*, estamos diciendo que queremos acceder al **recurso "pets"** y con el **verbo GET** indicamos que queremos recibir este recurso. De existir ese servicio, seguramente nos devolvería un listado de animales.

También **tenemos que indicar la versión del protocolo** (HTTP 1.0, HTTP 2.0) que estamos utilizando y **opcionalmente, una serie de cabeceras**. En las cabeceras se pueden indicar muchas cosas, como por ejemplo, si estamos solicitando un recurso que está securizado, podemos incluir en una cabecera nuestro token de identificación, que nos da acceso al recurso. También podemos indicar el **formato de respuesta** que vamos a aceptar, si es una página web o si es una serie de información en formato JSON o XML...




Http/2
HTTP 2.0
autentia
¿Qué es?
HTTP 2.0 es un protocolo binario que busca aumentar la velocidad y reducir la complejidad de las aplicaciones. Esto lo consigue cambiando el modo en el que se formatean los datos y se transportan.
CARACTERÍSTICAS
HTTP 2.0 conserva la semántica de HTTP 1.0, se mantienen sus conceptos básicos (verbos, códigos de estado, campos de cabecera, etc.) y se introducen mejoras con el objetivo de mejorar el rendimiento y optimizar nuestras aplicaciones.
De este modo no es necesario cambiar cómo las aplicaciones existentes funcionan, pero los nuevos desarrollos pueden aprovechar las funcionalidades que este protocolo ofrece.
HISTORIA
Se considera una evolución del protocolo SPDY, desarrollado por Google en 2009 y que también buscaba reducir los tiempos de carga de la web. Muchas de las mejoras que introducía SPDY fueron llevadas a HTTP 2.0.
En 2012 aparecen los primeros borradores de HTTP 2.0 y en los años siguientes ambos protocolos continuaron evolucionando en paralelo, hasta que en 2015 SPDY fué deprecado en favor de HTTP 2.0.
MEJORAS
Multiplexación: permite enviar y recibir varias peticiones al mismo tiempo usando una misma conexión. Así se consigue mejorar la velocidad de carga de la página y se disminuye la carga en los servidores web.
Protocolo binario en lugar de texto: ofrece mayor eficiencia en el transporte del mensaje y en su interpretación. Además son menos propensos a errores ya que con el texto hay problemas de espacios en blanco, capitalización, finales de línea, etc.
Servicio 'server push': el servidor envía información al cliente antes de que el cliente la pida. Así cuando el cliente necesite esos recursos ya los tiene, ahorrando tiempo.
Compresión de cabeceras: en general las cabeceras cambian poco entre peticiones y además con el uso de cookies su tamaño puede incrementar mucho. Por eso con HTTP 2.0 se van a enviar comprimidas.
Priorización de transmisiones: dentro de una misma conexión se da más peso a aquellas transmisiones que tengan más importancia para gestionar mejor los recursos.


Cabeceras HTTP más utilizadas
autentia
¿Qué es?
Algunas de las cabeceras HTTP más usadas son las que tienen que ver con el caché para mejorar el rendimiento y las que tienen que ver con seguridad para evitar incidentes como secuestros de sesión o ataques XSS (Cross-site scripting), etc.
DE CACHÉ
Cache-Control: esta cabecera se utiliza para especificar parámetros de cacheado en el navegador.
no-cache: obliga a validar un recurso caducado contra el servidor.
no-store: no permite cachear la respuesta. Muy usada en páginas con datos confidenciales.
private vs. public: si solo se permite la caché del navegador o también a terceros (CDN, Proxies, etc).
max-age: vida útil del recurso cacheado (en segundos).
Validators: utiliza la cabecera ETag como hash del recurso que tiene el servidor y verificar si el recurso cacheado caducado ha cambiado. Si no ha cambiado, no lo vuelve a solicitar, actualizando el recurso.
Extension Cache-Control: Estas son algunas cabeceras que extienden la directiva Cache-Control.
immutable: no valida con el origen, solo se refresca si caduca.
stale-while-revalidate: especifica el tiempo que se mostrará el archivo obsoleto mientra se valida en origen.
stale-if-error: especifica tiempo extra de un recurso caducado sin validar.
DE SEGURIDAD
HTTP Strict Transport Security (HSTS): solo se permite conexiones HTTPS. Esto evita incidentes de seguridad como el secuestro de cookies.
Content Security Policy (CSP): es una capa de seguridad adicional que ayuda a prevenir ataques de inyección de código.
Estableciendo whitelist de contenidos como js, images, font, etc.
Permite establecer políticas de navegación (a qué recursos se puede redirigir al usuario).
Podemos generar informes o limitar el uso de recursos como plugins.
Cross Site Scripting Protection (X-XSS): habilita un filtro en los navegadores contra ataques XSS.
X-Frame-Options: restringe el renderizado de objetos como <frame>, <iframe>, <embed> o <object> para prevenir los ataques de clickjacking.
X-Content-Type-Options: esta cabecera evita que el usuario pueda determinar el tipo de archivo que espera el servidor e intente camuflar bajo otro tipo de archivos, por ejemplo hacer pasar script de php como imágenes.



El protocolo HTTP cuenta también con lo que se conoce como **parámetros**. Generalmente se usan a modo de filtro. Veamos un ejemplo:

Se hace una llamada **GET** a la URL https://protectoradeanimales.com/api/v2/pets**?**breed=pitbull**&**color=brown

Los parámetros se indican al final de la url, justo después del símbolo de cierre de interrogación y se separan por el símbolo & si estamos enviando más de uno. Son pares de clave - valor. Lo que cabría esperar de esta consulta es que devolviera un listado de animales, de raza pitbull de color marrón.

A veces se utiliza la propia URL para hacer este tipo de filtros. Es muy normal ver peticiones de este tipo:

GET a la URL https://protectoradeanimales.com/api/v2/pets/**346**

Lo que se espera de esa llamada es que nos responda con los datos del animal cuyo identificador es el 346.

No nos tenemos que olvidar que hay veces que sólo con la URL no podemos completar todas las operaciones que se nos pueden ocurrir. Si

queremos dar de alta mascotas y queremos enviar los datos de la mascota al servidor, seguramente estemos haciendo algo parecido a lo siguiente:

POST a la URL https://protectoradeanimales.com/api/v2/pets

Las peticiones de tipo POST pueden ir acompañadas de un cuerpo en la petición. Para el ejemplo vamos a enviar el **body o cuerpo de la petición** en formato Json. Sería algo parecido a lo siguiente:

{

“name”:”thor”,

“breed”:”Pitbull”

“birthDate”:”21-07-2019”

}



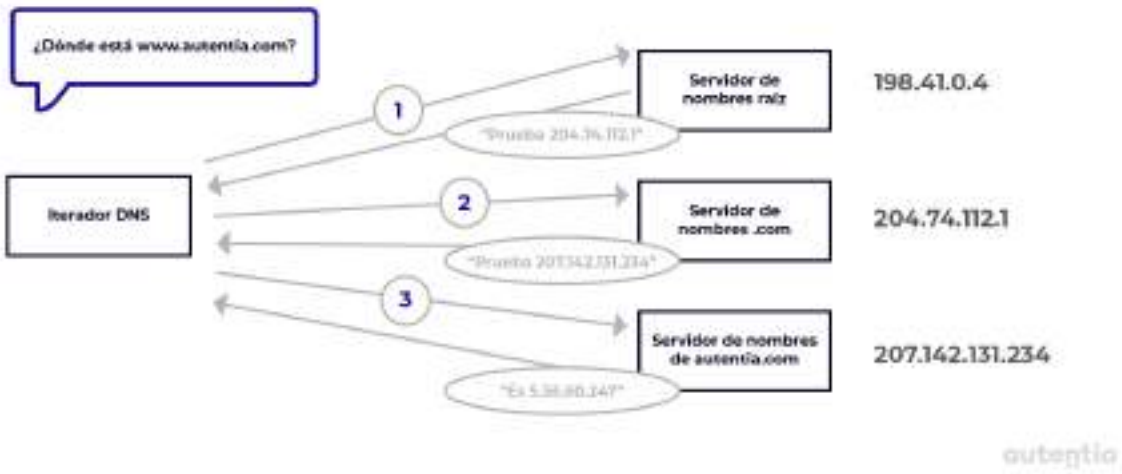
DNS

Una vez entendido esto, ¿cómo se conecta un dispositivo a un servidor web? ¿Cómo sabe a qué dirección IP debe realizar las peticiones? Ahí es donde entra en juego el protocolo **DNS** (Domain Name System). El DNS es un sistema de nombramiento jerárquico y descentralizado para ordenadores, servicios, etc., conectados a internet o a una red privada. Básicamente, es lo que **permite traducir un nombre fácilmente memorizable** (por ejemplo, www.google.com) **a una IP** (como puede ser 216.58.209.68). Si se copia y pega esa dirección IP en un navegador, se puede comprobar que corresponde al buscador.

Este sistema (DNS) se mantiene por un sistema de base de datos distribuida.



Cada dominio tiene al menos un servidor de nombres (names server) autorizado que publica información sobre el dominio y los servidores de nombres de cualquier dominio subordinado. Estos servidores autorizados son servidores que responden con IPs que no son cacheadas, sino que han sido instaladas en su sistema de configuración. De esta forma, cuando se escribe un dominio, se comienza un proceso de queries para recoger la IP del sitio. Primero, se pregunta al servidor de nombres raíz y este responderá con la IP del servidor de nombres subordinado correspondiente, que será el de la terminación del dominio (.org, .com, .es, etc.). Después, se preguntaría a éste y se recogería la IP del sitio web al que se quiere acceder.



Realmente este mecanismo significa una carga enorme en los servidores raíz, por lo que normalmente se utiliza una caché en los servidores DNS para evitarlo. Al final, estos servidores raíz sólo participan en una fracción muy pequeña de todas las peticiones. Esto es debido a que una vez que conocemos la IP, no volvemos a preguntar. Esta IP se almacena en la caché de nuestro ordenador.



HTTPS

HTTPS (Hypertext Transfer Protocol **Secure**) es una extensión de HTTP utilizada para una comunicación segura, identificando a los interlocutores (servidor y opcionalmente el cliente) y estableciendo una comunicación privada a través de internet. Para ello, **el protocolo de comunicación es encriptado utilizando TLS** (Transport Layer Security). Gracias a esta encriptación bidireccional, se puede determinar con seguridad que nos estamos comunicando con la página web que deseamos sin la interferencia de posibles atacantes. Esta encriptación consiste en una **encriptación simétrica** con unas claves que son generadas de forma única para cada nueva conexión, y están basadas en un secreto compartido que se negocia con el servidor al principio de la sesión, en lo que se denomina **TLS handshake**. El hecho de que sea una encriptación simétrica significa que ambas claves (la que utiliza el servidor y la que utiliza el cliente) son privadas e idénticas.

TLS

autentia

¿Qué es?

Transport Layer Security (TLS) es un protocolo criptográfico que proporciona autenticación y cifrado de la información intercambiada entre las distintas partes que operan sobre una red.

¿EN QUÉ CONSISTE?

TLS es el **sucesor de SSL (Secure Socket Layer)**. Se caracteriza por:

- **Comunicación privada y encriptada** tanto en el lado cliente como en el lado servidor a través de **un cifrado simétrico.**
- **Intercambio de claves** públicas y autenticación a través de **certificados digitales.**
- **Negociación del algoritmo** de intercambio de mensajes entre las partes.

TLS **intercambia registros** entre las partes en un **formato específico**. Cada registro es comprimido, cifrado y empaquetado con un código de MAC. Hay dos protocolos para ello:

- **TLS Handshake:** es un protocolo que sirve para que dos partes se verifiquen entre sí y puedan establecer un tráfico cifrado e intercambiar claves. Las claves generadas para dicho cifrado se consiguen a través de una negociación entre las partes.
- **TLS Record:** se lleva a cabo la autenticación de los datos para que su transmisión sea mediante una conexión fiable y segura (p.e. TCP). Los mensajes que se intercambien estarán cifrados simétricamente mediante las claves negociadas en el Handshake.



Cifrado simétrico

autentia

¿Qué es?

El **cifrado simétrico** es una forma de encriptación en la que sólo se utiliza una clave, tanto para encriptar como para desencriptar.

¿EN QUÉ CONSISTE?

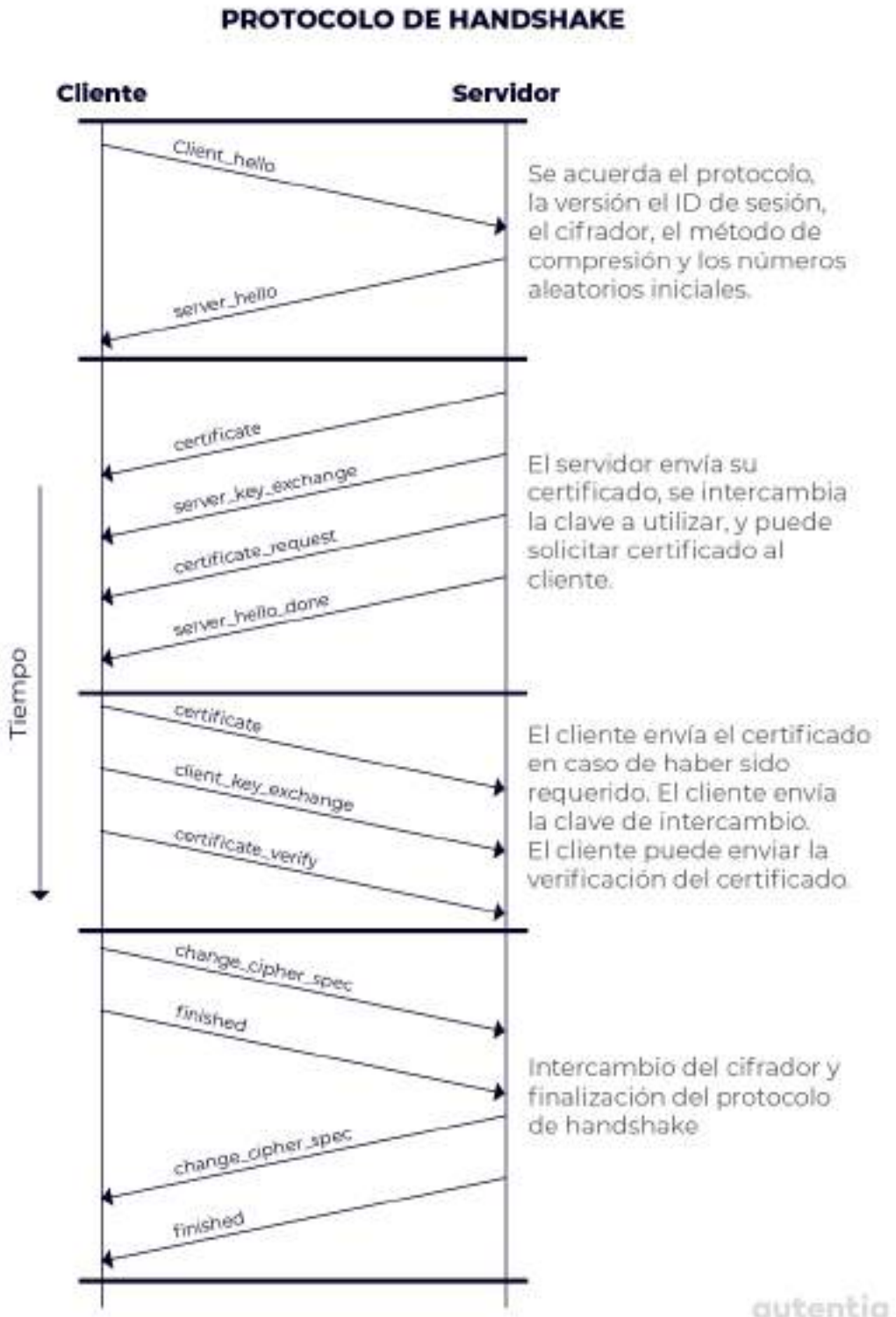
Al haber sólo una clave, esta es **privada**, y las distintas entidades comunicándose **deben compartirla entre ellas** para poder utilizarla en el proceso de desencriptación.

El hecho de que todas las partes tengan acceso a la clave privada es la **principal desventaja** de este tipo de encriptación, ya que hay más probabilidad de que esa clave pueda ser vulnerada. Además, si un atacante consigue esta clave podría desencriptar y leer todos los mensajes y datos de esa conversación. La **ventaja** que tiene sobre la alternativa de usar un cifrado asimétrico es que tiene mejor rendimiento, porque los algoritmos usados son más sencillos.

La clave puede ser una contraseña/código o puede ser una cadena de texto o números aleatoria generada por un software especializado en generar este tipo de claves.



Sin entrar en mucho detalle, el protocolo de handshake donde se negocia esa clave tendría un flujo como el de la imagen.



Aparte de esta encriptación, para confirmar que el servidor es de confianza, el protocolo HTTPS requiere de una autenticación por parte de un tercero de confianza (una **autoridad certificadora**) que firme certificados digitales. Cuando una entidad solicita un certificado, la autoridad certificadora (AC) verifica la identidad del solicitante (ya sea mediante carnés de identidad, huellas dactilares, declaraciones juradas...) y después crea una clave pública (para cifrar los mensajes) y una privada (para descifrarlos). El uso para el cifrado de la clave pública y privada se conoce como **cifrado asimétrico**. Los navegadores suelen incluir por defecto un repositorio de AC de confianza, pero si hay alguna AC en la que en principio no se confía (a lo mejor porque todavía no lleva mucho tiempo en funcionamiento), habrá que instalar manualmente el certificado en el repositorio del navegador. Es importante conocer que **estos certificados caducan**. Una iniciativa interesante desde el punto de vista de devops es alertar sobre la caducidad de los mismos para que así, cuando caduque, no nos quedemos sin peticiones. Esto suele pasar mucho en el entorno profesional. El certificado se renueva, y como no caduca hasta dentro de 1 año o X tiempo, todo el mundo se olvida de ello, hasta que un día, de repente, no funciona y ya tenemos el problema montado en producción. Además **es una buena práctica tener estos certificados en un repositorio** para poder instalarlos en los servidores que se necesite de forma automática.

Certificado

autentio

¿Qué es?

Documento virtual que contiene los datos identificativos de una persona física o de un sitio web y que está autenticado por un organismo oficial encargado de emitir y validar dicha información.

TIPOS

Cuando queremos identificar a una persona física, una entidad o un dominio de manera digital, hablamos de **certificados digitales**. Una **Autoridad Certificadora (AC)** será la encargada de firmar y emitir los certificados, verificando que quien lo solicite es quien dice ser.

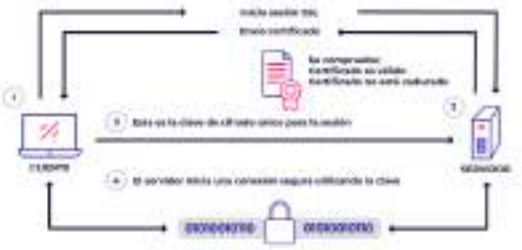
También podemos validar un sitio web a través de certificados. Se configura el servidor para que use dicho certificado y usando el protocolo SSL se cifra la información enviada al servidor. Esto es lo que comúnmente se conoce como **certificado SSL** debido a que este protocolo es el más extendido hoy en día. Los navegadores suelen incluir por defecto un repositorio de AC de confianza, pero si hay alguna AC en la que en principio no se confía (porque lleva poco tiempo en funcionamiento, por ejemplo), habrá que instalar manualmente el certificado en el repositorio del navegador. **Los certificados caducan** y esto se hace para asegurar que toda la información es precisa y demuestra una validez como propietario de confianza del dominio.

¿Qué ventajas nos ofrecen los certificados?

- Ahorro de espacio, tiempo y dinero.
- Confidencialidad y seguridad en las comunicaciones.

¿CÓMO FUNCIONA?

Se hace uso de claves públicas y claves privadas. En el caso de un sitio web, la clave privada permanece en el servidor donde se ha configurado el certificado. Cuando un navegador web desea establecer una nueva conexión, el servidor comparte la clave pública con el cliente para establecer un método de cifrado y el cliente confirma que reconoce y confía en la entidad emisora del certificado. Este proceso inicia una sesión segura que protege la privacidad y la integridad del mensaje.



Cifrado asimétrico

autentio

¿Qué es?

El **cifrado asimétrico** es una forma de encriptación en la que las claves vienen en parejas. Lo que una de las llaves encripta, sólo puede ser desencriptado por la otra.

¿EN QUÉ CONSISTE?

El cifrado asimétrico tiene **dos claves** por persona, a diferencia del cifrado simétrico que sólo tiene una. Pongamos que tenemos dos personas: Bob y Alice. Los pasos para encriptar y desencriptar un mensaje que Bob le envía a Alice serían los siguientes:

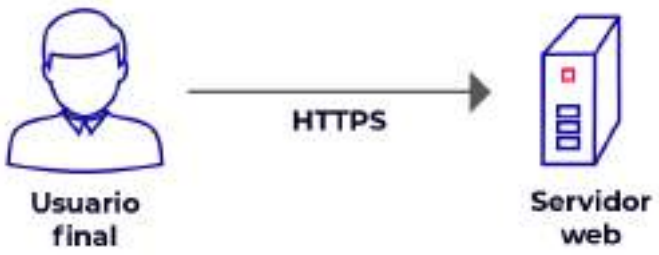
- Bob dispone de la clave pública de Alice, que sirve para encriptar (las claves públicas las puede ver cualquiera). **Encripta un mensaje con esa clave pública**.
- **Manda el mensaje encriptado a Alice**.
- Los mensajes que se hayan encriptado usando la clave pública de Alice sólo se pueden desencriptar usando la clave privada de Alice, clave que solo ella conoce. **Alice usa su clave privada para desencriptar el mensaje**.

A TENER EN CUENTA

Si un atacante consiguiera la clave privada de Bob podría leer los mensajes que le llegan a Bob, ya que han sido encriptados usando su clave pública, **pero no podrían descifrar mensajes que Bob mande a otros**, ya que estos mensajes se encriptan usando claves públicas de otros.



Ahora que ya hemos aclarado qué es HTTP y HTTPS, es conveniente aclarar que los servidores web típicamente abren 2 puertos. El **puerto 80** de un servidor se utiliza para las comunicaciones HTTP y el **puerto 443** para las comunicaciones HTTPS. Estos puertos no se ponen de forma aleatoria, sino que **están estandarizados** para esta función. Puede verse en el siguiente enlace. En muchas ocasiones y por seguridad, las comunicaciones HTTP están deshabilitadas y lo que se hace es que se redirige el puerto 80 al 443 para permitir únicamente establecer comunicación HTTPS.

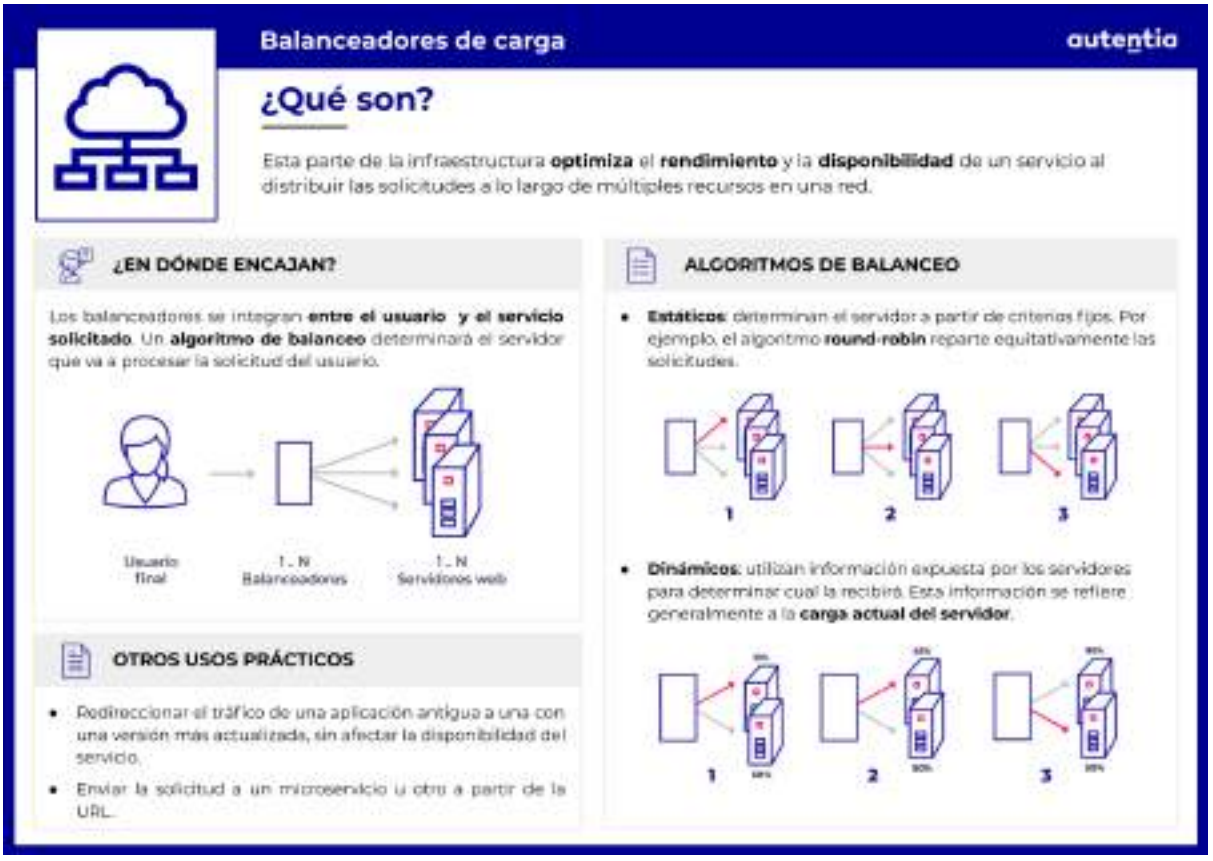


¿Qué pasa cuando con un solo servidor no somos capaces de dar abasto porque recibimos muchísimas peticiones?, ¿cómo es posible poner otro? Hasta ahora hemos visto que por DNS si llamamos a https://protectoranimales.com esto va a resolver a una IP del tipo 158.34.56.230. ¿Cómo hacemos si queremos poner 2 servidores web o más? Pues **para eso existen los balanceadores que vamos a ver a continuación**, para apuntar el DNS a un balanceador por delante de los servidores web y no directamente a los servidores web.


DNS
autentia
¿Qué es?
El DNS (Domain Name System) es un sistema de nombramiento jerárquico y descentralizado para ordenadores, servicios, etc. conectados a internet o a una red privada. Permite traducir un nombre fácilmente memorizable para una persona a una IP.
CONCEPTO
El DNS nació de la necesidad de recordar fácilmente los nombres de los servidores de Internet. Cada dominio tiene al menos un servidor de nombres que publica información sobre ese mismo dominio y sobre los servidores de nombres de cualquier dominio subordinado.
Cuando intentas acceder a www.autentia.com comienza un proceso de consultas iterativo para resolver la IP del sitio.
1. Primero se pregunta al servidor de nombres raíz, que responde con la IP de un servidor de nombres subordinado.
2. El servidor de nombres subordinado responde con la dirección del servidor de nombres de autentia.com.
3. Finalmente este servidor conoce la IP del dominio y lo devuelve.
En general, la resolución de los nombres se hace de forma transparente al usuario a través del cliente (navegador, aplicación de correo, etc.).
Además, para reducir el número de peticiones, ya que las direcciones se cachean a distintos niveles. Por ejemplo los navegadores cachean los resultados de estas consultas y también los servidores de nombres disponen de cachés para evitar hacer tantas peticiones.
1
2
3
198.41.0.4
204.74.112.1
207.142.131.234

Balanceadores

Los balanceadores son la parte de la infraestructura que se va a encargar de **distribuir la ejecución de procesos y tareas** entre piezas iguales, de tal forma que no haya algunos servidores ociosos mientras otros están muy cargados, dando tiempos peores de finalización de tareas o de respuesta. Se utilizan para **optimizar el uso de recursos hardware**.



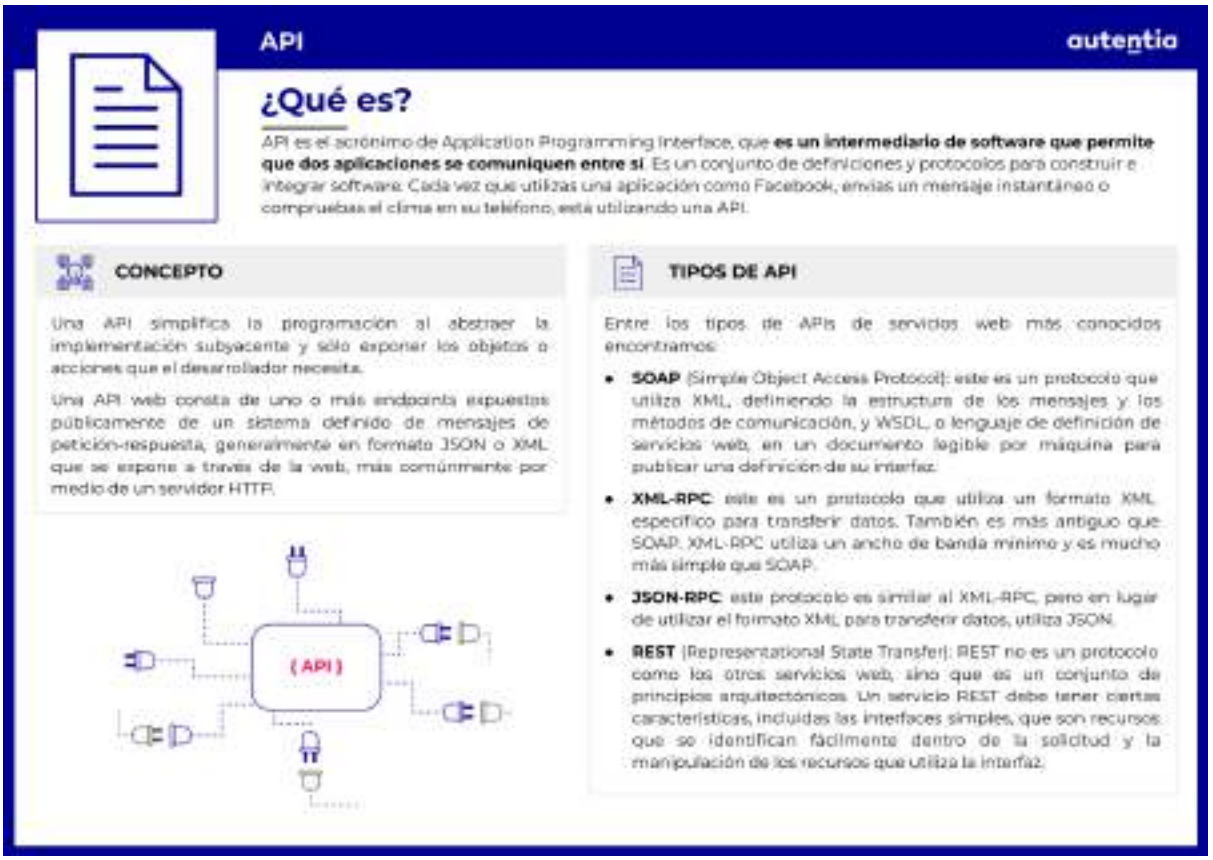
Son entonces una **pieza básica para el rendimiento**.

También son una **pieza importante** en cuanto a la **alta disponibilidad**. Mediante los balanceadores podemos dejar de apuntar a servidores con una aplicación antigua y apuntarlos hacia servidores con una nueva versión de nuestra aplicación. Sirven en este caso para **redireccionar tráfico**.

En algunos casos, hay balanceadores, por ejemplo los de AWS, que tienen **mecanismos de health check**, esto es, comprobar que los servidores a los que están mandando tráfico o tareas, efectivamente se encuentran

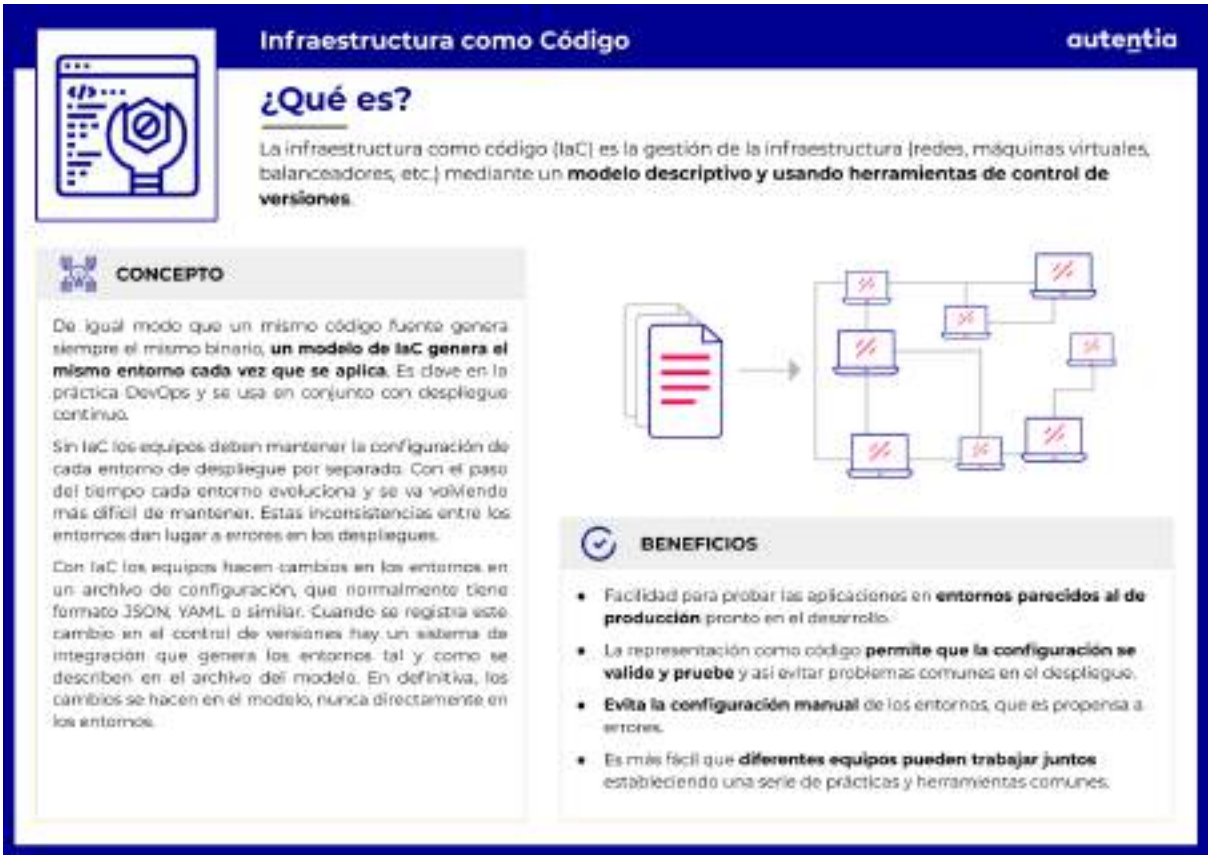
operativos y funcionando. Si no fuera este el caso, a partir de ese momento no enviarían más tareas a esos servidores que no responden.

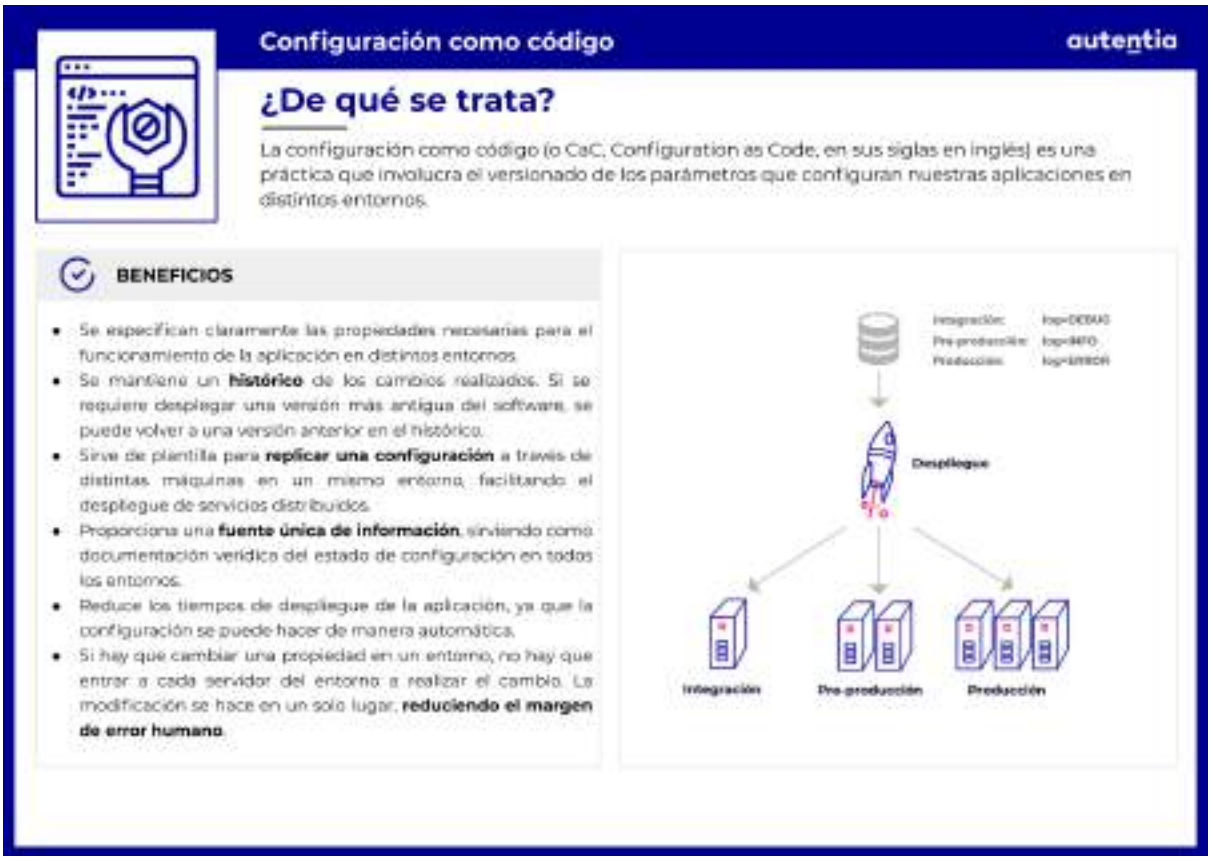
Cuando veamos más adelante los servidores de aplicaciones y cómo éstos son los encargados de exponer APIs pueden surgir dudas. Si los balanceadores son el punto de entrada: ¿cómo vamos a discernir cuándo una petición es para pedir una página estática, por ejemplo a un NGINX, y cuándo para acceder a un API que tenemos en un Tomcat? Estos balanceadores **muchas veces redirigen también entre aplicaciones o microservicios por la url**. En función de una url le mandan la petición a un conjunto de máquinas o a otro.



Hay multitud de software que puede hacer estas tareas de balanceo, pero casi todos comparten una cosa en común, **requieren configuración**. Es muy importante que seamos capaces de cambiar esta configuración en cualquier momento, de volver a una versión anterior... y todo ello de **forma automática**. Es normal tener una infraestructura con muchos balanceadores a lo largo de todos los entornos que hemos ido viendo con anterioridad. Ir modificando esta infraestructura o cambiando esa configuración a mano para los distintos entornos, para tener todos los balanceadores actualizados y configurados, es garantía de que las cosas

van a salir mal y vamos a tener errores. Es por eso que es importante automatizar estas tareas y para ello, existen herramientas como IaC (*Infrastructure as Code*) y CaC (*Configuration as Code* referenciados en la parte uno de este documento).





Es importante también saber que hay distintos algoritmos que se pueden utilizar en los balanceadores. Se dividen en dos grande bloques:

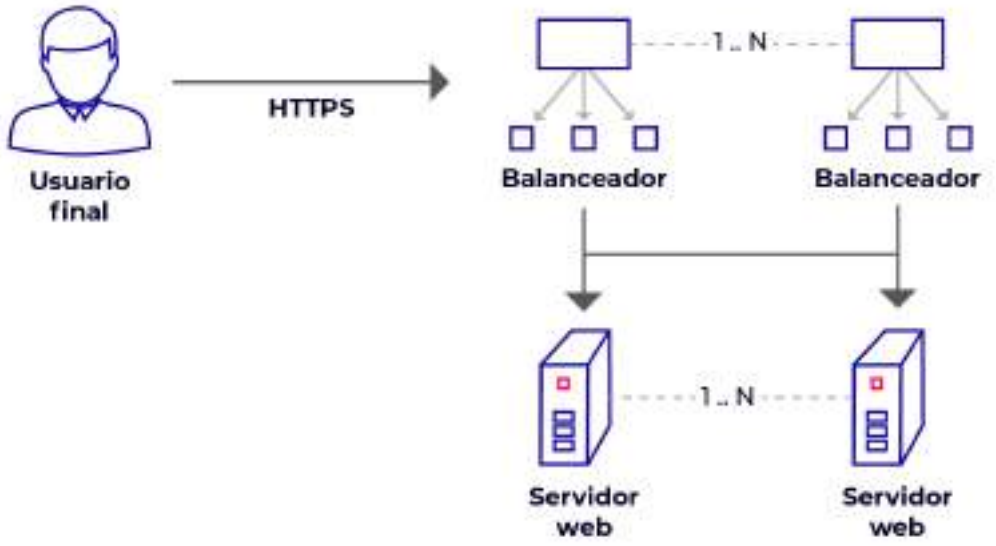
1. **Algoritmos estáticos**: **no tienen en cuenta la carga de las máquinas** a las que envían las peticiones o tareas. Da igual que una máquina esté muy cargada, si le toca recibir tarea, la va a recibir. El algoritmo más común es **round robin**, o sea, si tengo 4 máquinas, le envío por orden primero a la 1ª, acabando por la 4ª y la 5ª tarea se la vuelvo a mandar al primer "nodo". Se conoce como nodo a cada uno de los servidores que forman un conjunto que recibe tareas de un balanceador. Suelen estar agrupados por funcionalidad o similitud de tareas. Este tipo de algoritmos tiene la ventaja de que es **muy sencillo de implementar**, y además, **para ciertos escenarios**, como puede ser el de responder peticiones http, **es muy eficiente**.

2. **Algoritmos dinámicos**: estos algoritmos, al contrario que los anteriores, **sí tienen en cuenta la carga de los nodos** a los que balancean. **Son muy útiles cuando las tareas que se tienen que distribuir varían mucho de unas a otras**, que es lo que puede causar la congestión de un nodo, frente a otro mucho más libre. En estos casos, aplicar estos algoritmos tiene unos resultados excelentes. Un

ejemplo puede ser el procesamiento en batch de lotes de tareas con tamaño dinámico.

Una restricción importante es también la necesidad en algunos sistemas de balancear las peticiones con mismo origen, siempre hacia el mismo servidor destino, vinculando dichas peticiones a las sesiones del usuario. A esto se le denomina “afinidad de sesión” o “sticky session”.

Es frecuente encontrarse servidores web, como puede ser Nginx o Apache, haciendo de balanceadores, pero también hay software específico para realizar esta función.



Servidores de aplicaciones

Aparte de los web servers, existen otro tipo de servidores: los app servers o servidores de aplicaciones. Estos están ligados a la **lógica de negocio** y **son más complejos** que los web servers.

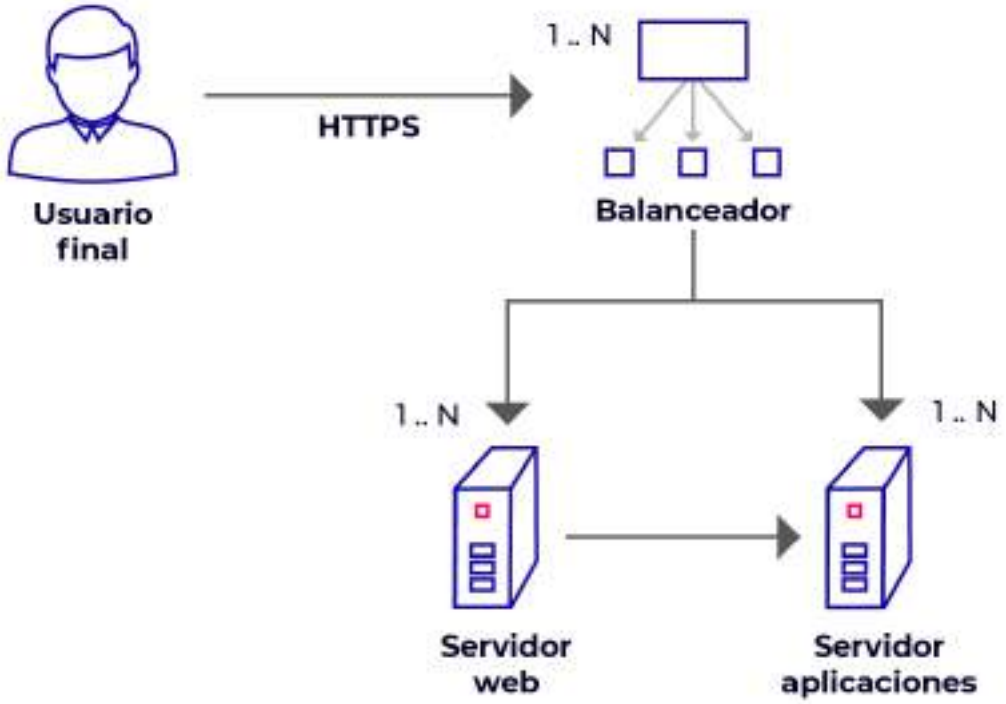
Si bien la tarea principal de los **web servers** es **servir páginas web estáticas**, los **app servers** sirven aplicaciones. Estas aplicaciones se pueden categorizar en varios tipos entre los que se encuentran:

- **Aplicación web dinámica**, la cual es servida a un navegador.
- **API (REST, SOAP, etc.), que expone los recursos disponibles para ser consumidos por los clientes** y realiza operaciones ligadas al negocio. Por ejemplo, si estás utilizando la app de Instagram y la aplicación quiere acceder a la información de tu perfil para pintarla, realizará una petición a un app server.

Mientras que los clientes de un webserver son navegadores, los de un app server pueden ser o bien un navegador, o bien **otros web servers que necesitan** cierta **información antes de servir una página web o responder una petición** o incluso, **otros app servers cuyas aplicaciones necesiten información**.

La conexión a estos servidores se puede realizar tanto por HTTP como por HTTPS, utilizando puertos distintos a los de los servidores web. El protocolo a utilizar dependerá de **si estamos en una red interna de confianza**, donde se puede utilizar **HTTP** o **incluso de la sensibilidad del dato**, que aunque sea una red interna, puede decidirse comunicarse mediante **HTTPS**. **Si la comunicación traspasa nuestra red interna** es recomendable utilizar **HTTPS**.

Algunos de los servidores de aplicaciones más utilizados son Apache Tomcat, Jetty, JBoss Wildfly, IBM Websphere y Liberty, entre otros.


1 .. N
HTTPS
Usuario final
Balanceador
1 .. N
1 .. N
Servidor web
Servidor aplicaciones

autentia

Sistemas de bases de datos

Un **sistema de Base de Datos** (**DB** por sus siglas en Inglés) es una herramienta que **captura, organiza y relaciona datos,** permitiendo al usuario, ya sea una persona u otro sistema, **acceder, analizar y/o explotar estos datos**.

Las bases de datos se pueden clasificar de muchas formas según el contexto que se esté manejando, la utilidad de las mismas o las necesidades que satisfagan. Una de las formas de clasificación más utilizada dentro de contexto de TI es la **clasificación según modelo de administración de datos**. Algunas de las categorías más destacadas son:

- **Relacionales**, donde los datos se almacenan y se acceden por medio de relaciones ya establecidas. Estos datos se organizan en un conjunto de **tablas** con filas y columnas. Las tablas se utilizan para contener información sobre los objetos que se representarán en la base de datos. Cada **columna** de una tabla contiene un cierto **tipo de dato** y un campo almacena el valor real de un atributo. Las **filas** en la tabla representan una colección de **valores relacionados de un objeto o entidad**. Cada fila de una tabla se puede marcar con un identificador único llamado clave principal, y las filas entre varias tablas, se pueden relacionar utilizando claves externas. Se puede acceder a estos datos de muchas maneras diferentes sin reorganizar las tablas de la base de datos. El lenguaje más común utilizado en este tipo de bases es **SQL** (Structured Query Language). Algunas de las DB más conocidas de esta categoría son PostgreSQL, MySQL, Oracle y MariaDB.

- **No Relacionales** o NoSQL, son el conjunto de todas aquellas bases de datos que proporcionan un mecanismo para el almacenamiento y la recuperación de datos que se modelan en medios distintos de las relaciones tabulares utilizadas en las bases de datos relacionales. Este tipo de base de datos tiene aplicaciones diversas según su naturaleza, desde cachés distribuidas a almacenamiento primario de datos.

Base de Datos

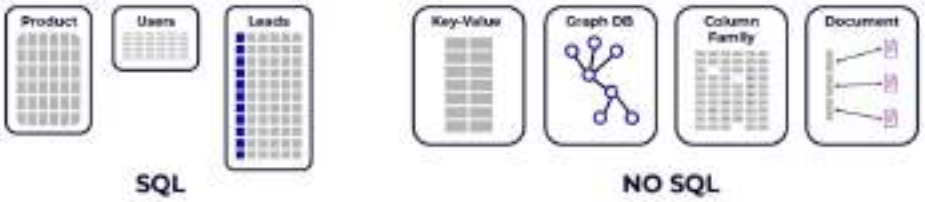
autentio

¿Qué es?

Una base de datos es una especie de "almacén" que nos permite guardar grandes cantidades de información para su posterior **recuperación, análisis y/o transmisión**. Podemos encontrar diversos tipos de bases de datos y se clasifican de acuerdo a las necesidades que busquen solucionar en dos grandes bloques.

TIPOS

No Relacionales	Definición	Ejemplo
Documentales	**Los datos se almacenan y se consultan como documentos tipo JSON**. Los documentos pueden contener muchos pares diferentes clave-valor, o incluso documentos anidados. Son más flexibles al cambio ya que si el modelo de datos necesita cambiar, solo se deben actualizar los documentos afectados y no todo el esquema.	MongoDB
De Grafos	**Usan nodos para almacenar entidades de datos y aristas para almacenar las relaciones entre nodos**. El valor de estas bases de datos se obtiene de las relaciones entre nodos y no hay un límite para el número de relaciones que un nodo pueda tener. Un borde siempre tiene un nodo de inicio, un nodo final, un tipo y una dirección.	Neo4J, HyperGraphDB



Bases de Datos NoSQL

autentio

Definición

Las bases de datos noSQL manejan grandes cantidades de información no estructurada, almacenando los datos de diferentes formas (documentos, grafos o colecciones de clave-valor, etc.)

VENTAJAS

- Asegura que **la base de datos no se convierta en un cuello de botella** si la cantidad de datos aumenta.
- **Almacena grandes cantidades de datos desestructurados**, pudiendo almacenar cualquier dato sin establecer restricciones por su tipo o estructura.
- Las bases de datos NoSQL **permiten un escalado horizontal en múltiples nodos o servidores** de forma inmediata y sin problemas.
- Debido a su naturaleza no relacional, **no necesita un modelo de datos muy detallado, lo que ahorra tiempo de desarrollo**.

DESVENTAJAS

- La comunidad **NoSQL carece de madurez**, ya que es relativamente nueva frente a los RDBMS de SQL.
- La falta o **escasez de herramientas para generar informes** sobre el análisis y pruebas de rendimiento en noSQL frente a la amplia gama que encontramos en SQL.
- En el lenguaje de consulta en las bases de datos noSQL usa sus propias características, por lo que **no es 100% compatible con el lenguaje SQL** utilizado en las bases de datos relacionales.
- Hay muchas bases de datos NoSQL y **una falta de estandarización** en ellas, pudiendo causar un problemas en las migraciones.

Algunas de las bases de datos noSQL más utilizadas son:

- Redis
- Neo4j
- CouchDB
- Postgres
- Hbase
- MongoDB
- BigTable
- Cassandra

Esta categoría a su vez comprende otras categorías. Las más destacadas son:

- **De tablas, columna ancha o BigTables**, donde se guardan los datos a lo largo de tablas que pueden tener muchísimas columnas. Se suele usar para guardar búsquedas en internet. Ejemplos son Cassandra, HBase...
- **Orientada a Grafos**, donde se representa la información como nodos de un grafo y sus relaciones con las aristas del mismo, de manera que se pueda usar teoría de grafos para recorrer la base de datos ya que ésta puede describir atributos de los nodos (entidades) y las aristas (relaciones). Algunas de las DB más conocidas de esta categoría son Neo4j y Sparksee.
- **Clave-Valor**, donde se almacenan datos como un conjunto de pares clave-valor en los que una clave sirve como un identificador único. Tanto las claves como los valores pueden ser cualquier cosa, desde objetos simples hasta objetos compuestos complejos. Algunas de las DB más conocidas de esta categoría son Redis o Dynamo.
- **Documentales**, que permiten almacenar y consultar datos como documentos de tipo JSON, YAML, XML y otros. En este enlace, podemos aprender cómo funciona una base de datos de este tipo, en este caso **CouchDB**. Las bases de datos de documentos facilitan a los desarrolladores el almacenamiento y la consulta de datos en una base de datos mediante el mismo formato de modelo de documentos que emplean en el código de aplicación. Algunas de las DB más conocidas de esta categoría son MongoDB, CouchDB, SimpleDB o Lucene.

A su vez, las bases de datos pueden ser **centralizadas** (toda la información de la base de datos se encuentra almacenada en un único lugar) o **distribuidas**, que son un conjunto de múltiples bases de datos lógicamente relacionadas y que se encuentran distribuidas en **diferentes espacios lógicos y/o geográficos** e **interconectados** por una red de comunicaciones. Dichas DB tienen la capacidad de realizar **procesamientos autónomos** y permiten realizar operaciones locales o distribuidas.

También se utilizan para Business Intelligence (BI o inteligencia de negocio), que comprende las estrategias y tecnologías utilizadas por las empresas para el análisis de datos de información empresarial. Dos de las formas

más comunes de implementar BI es mediante el uso de sistemas de Data Warehouse (DW o almacén de datos) y Data Lake.

- **Data Warehouse** es un sistema utilizado para reportes y análisis de datos. Son almacenes centralizados de datos integrados de una o más fuentes dispares. Almacenan datos actuales e históricos en un solo lugar que se utilizan para crear informes analíticos para los trabajadores de toda la empresa.

  Los datos almacenados en el almacén se cargan desde los sistemas operativos (como marketing o ventas). Los datos pueden pasar a través de un almacén de datos operativos y pueden requerir la limpieza de datos para operaciones adicionales para garantizar la calidad de los datos antes de ser utilizados en el DW para la presentación de informes.

- **Data Lake** es un sistema o depósito de datos almacenados en su formato natural (o en crudo), generalmente objetos de tipo blob o archivos. Un lago de datos suele ser un único almacén de todos los datos de la empresa, incluidas las copias sin procesar de los datos del sistema de origen y los datos transformados que se utilizan para tareas como informes, visualización, análisis avanzado y aprendizaje automático. Un lago de datos puede incluir datos estructurados de bases de datos relacionales (filas y columnas), datos semiestructurados (CSV, registros, XML y JSON), datos no estructurados (correos electrónicos, documentos y PDF) y datos binarios (imágenes, audio y vídeo). Se puede establecer un lago de datos on-premise (dentro de los centros de datos de una organización) o en la nube (utilizando servicios en la nube de proveedores como Amazon, Google y Microsoft).

Como podemos ver, las DB almacenan **datos vitales**, no sólo para las aplicaciones, si no para el negocio. Suelen recopilar, por ejemplo, toda la información interesante de los clientes, los potenciales o los ya consolidados, así como aquellos que dejaron de utilizar tus servicios. Es muy útil al poder realizar muchas acciones, como por ejemplo segmentar en base a intereses o analizar comportamientos.

Es por esto que es importante, cuando se trata de algo tan delicado como una base de datos, tener su versionado controlado. Si no, algo como introducir o eliminar una columna en una tabla se puede convertir en un desastre. Por eso existen **herramientas de control de cambios** de bases de



datos, como pueden ser Liquibase o Flyway.

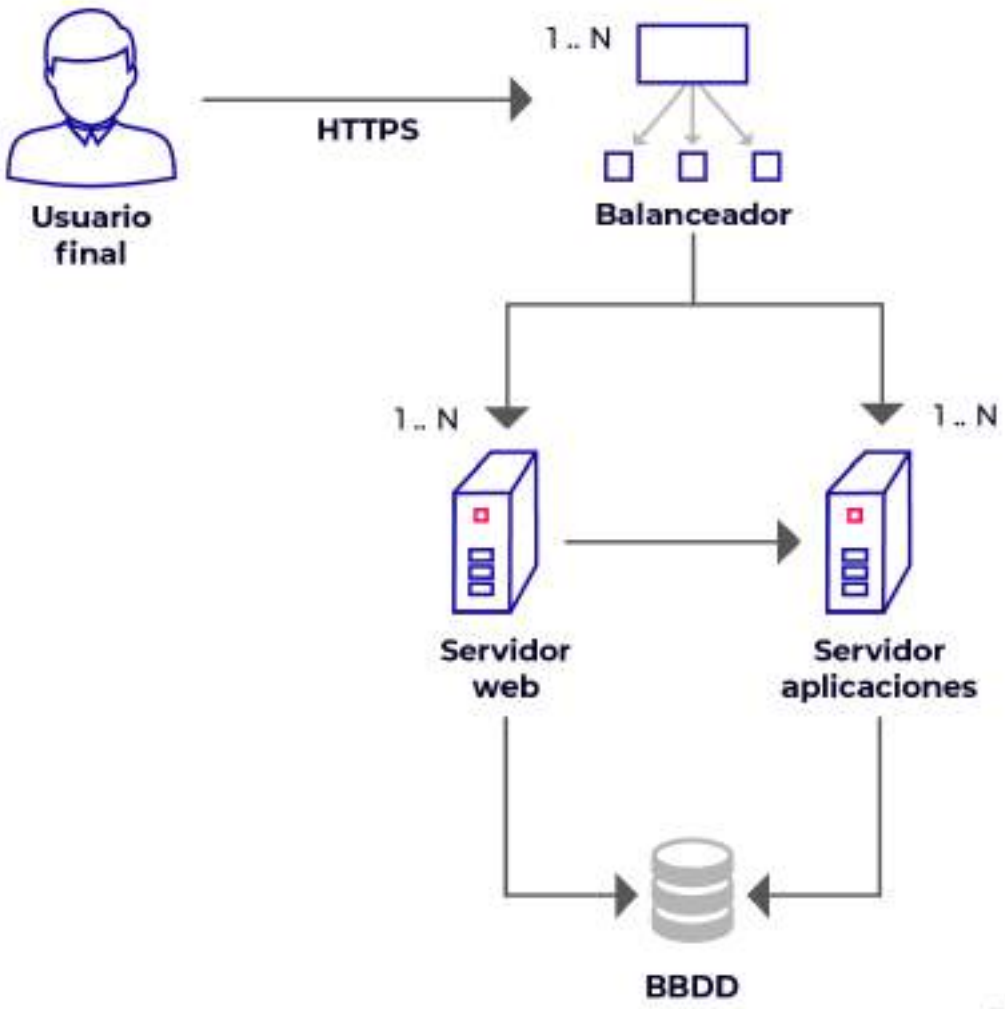


Pongamos, por ejemplo, que tenemos una base de datos de usuarios en la que tenemos los campos “nombre”, “dirección” y “correo electrónico”. Cuando ya llevamos en producción un tiempo, vemos que sería beneficioso guardar también el número de teléfono y decidimos añadir ese campo a la tabla. Si esto lo hiciéramos **a mano**, lo más probable es que **acabe en desastre**. Sin embargo, si usamos estas herramientas que ofrecen la posibilidad de **ir versionando estos cambios**, tendremos más **seguridad de que el cambio funcionará de la forma esperada**. Además, **si hay algún problema** al realizar algún cambio, gracias a estas herramientas podríamos simplemente **volver a la versión anterior** de la base de datos y **deshacer el cambio**.

También, gracias a que **tenemos guardados todos los cambios** que hemos ido realizando, si en algún momento se **crea una base de datos nueva**, estas herramientas lo detectarán y **generarán automáticamente todas las tablas** con todos los cambios que hemos realizado, sin necesidad por nuestra parte de ejecutar **ningún script a mano**.



En lugar de Liquibase o Flyway, se podrían también **idear alternativas** desde DevOps como un **proyecto de bbdd** donde se tengan todos los **scripts almacenados en un repositorio por versión y entorno**.



Sistemas de caché

Cuando tenemos un servidor expuesto a un gran número de peticiones, una técnica muy común y eficaz es utilizar un sistema de caché. Una caché es básicamente una **capa de almacenamiento de datos de alta velocidad** que **mantiene datos normalmente temporales** o de naturaleza transitoria, de modo que las **solicitudes de acceso** a esos datos **se atiendan más rápido que si se recogieran de la ubicación principal** del almacenamiento de esos datos. De esta forma, si hay ciertas peticiones que se **repiten mucho** y la **respuesta es siempre la misma**, esa **respuesta** se puede **cachear**, consiguiendo, entre otras, las siguientes ventajas:

- Por una parte, la **respuesta a esa petición será más rápida** que si se tuviera que acceder a la base de datos.
- Se **libera de carga al servidor**, pudiendo **dedicarse a otras peticiones** y por tanto, soportando una mayor carga de peticiones en general.

Cuando utilizamos un sistema de caché, es necesario **saber qué es lo que estamos haciendo** y tener en cuenta distintos aspectos como puede ser el **tiempo de vida** de los datos en la caché antes de eliminarlos (**TTL o Time to live**) o saber la **volumetría** de lo que vamos a cachear, es decir, **cuánto espacio** vamos a tener que utilizar según el **peso de los objetos** que vamos a querer guardar (MB, GB). Esto nos ayudará a decidirnos entre los distintos tipos de caché según la situación.

Los tipos de caché se pueden dividir en dos tipos: **no distribuida y distribuida**. La primera se suele utilizar para guardar resultados de operaciones que son **poco pesados**, siendo un ejemplo de esto Ehcache. En el caso de una caché **distribuida**, sin embargo, en vez de utilizar la memoria de la misma máquina, se dedica un **conjunto** de máquinas explícitamente para este fin. Un ejemplo de esto puede ser Redis. Una de las **desventajas** que suelen tener las **cachés distribuidas** es que su acceso es **más lento** que una en memoria, ya que normalmente tendremos que acceder a una **máquina externa**. Por otro lado, este tipo de caché es capaz de guardar **más información**, y por tanto es más útil para almacenar **resultados de operaciones más pesadas**. Por lo tanto, cuando vamos a cachear algo hemos de sopesar, según distintos criterios, en qué caché guardamos esta información.